SIDE PROJECT · AUTOMATION & MONITORING

```
if np.all(original == duplicate):
    continue
else:
    alert()
```

Website Change Monitor

A Python script that watches a webpage around the clock and fires an alert, screenshot attached, the moment something changes.

Python | Selenium | OpenCV | NumPy | Discord API

THE IDEA

Checking a page by hand, every day, is exactly the kind of work that belongs to a machine.

Some pages have no feed, no notification, no API: the only way to know if something changed is to open them and look. This script does that instead, in a continuous loop: it opens the page in a headless browser, takes a screenshot, compares it against the previous one pixel by pixel, and if it finds a difference, it fires an alert on Discord with the image attached.

01

Automatic screenshots

Headless Chrome via Selenium, no graphical interface required.

02

Pixel-by-pixel comparison

OpenCV and NumPy compare the two images in a single vectorized operation.

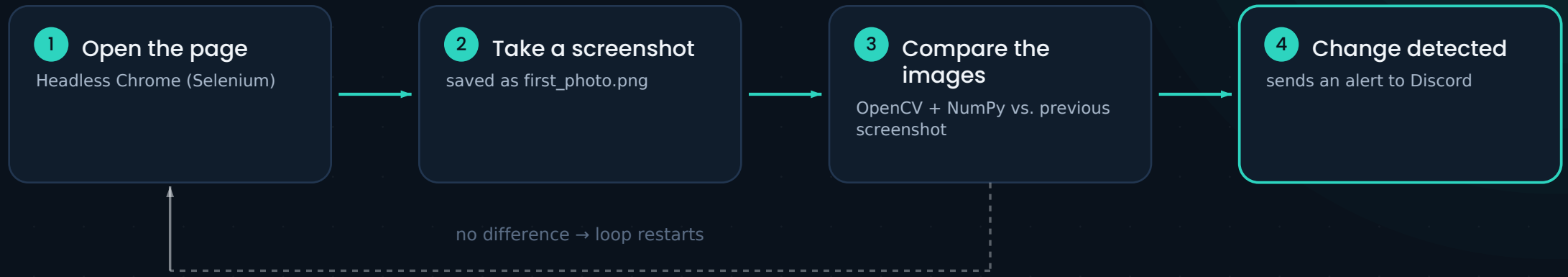
03

Instant notification

A Discord message with the screenshot attached, the moment something changes.

ARCHITECTURE

A simple loop, built to run unattended



In production the loop runs 24/7: every iteration opens and closes the browser, so it never holds a state of the site older than a few seconds.

TECHNICAL CHOICES

Why it's built this way

Screenshots instead of an HTML diff

Many modern pages load content via JavaScript: comparing raw HTML would give false positives. A screenshot sees the page exactly the way a person would.

OpenCV for the comparison

np.all(original == duplicate) compares two pixel matrices in a single shot: the fastest way to know if two images are identical.

A headless browser

No graphical interface: the script can run on a server, a Raspberry Pi, or inside a container, with no desktop needed.

Discord notifications

A webhook or a bot token is enough to send a message with the image attached: free, instant, and it lands straight on your phone.

SOURCE CODE

website_change_monitor.py

part 1 / 2

website_change_monitor.py

```python
import cv2
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
import time
import shutil
import requests
import numpy as np


def alert():
    payload = {
        "content": "The site has been updated"
    }
    header = {
        "authorization": "YOUR_DISCORD_TOKEN"
    }
    file = {
        "file": ("first_photo.png", open("first_photo.png", "rb"))
    }
    r = requests.post("https://discord.com/api/v9/channels/YOUR_CHANNEL_ID/messages", data=payload, headers=header, files=file)


url = "https://example.com/page-to-monitor"

while True:
    options = Options()
```

SOURCE CODE

website_change_monitor.py

part 2 / 2

website_change_monitor.py

```
    options.add_argument("--disable-gpu")
    options.add_argument("--headless")
    options.add_argument("--no-sandbox")
    options.add_argument("--disable-dev-shm-usage")
    service = Service(r"path/to/chromedriver")
    browser = webdriver.Chrome(service=service, options=options)
    browser.get(url)
    time.sleep(5)
    browser.save_screenshot("first_photo.png")
    browser.quit()

    original = cv2.imread("first_photo.png")
    duplicate = cv2.imread("second_photo.png")

    # difference = cv2.subtract(original, duplicate)
    # print(difference)

    if np.all(original == duplicate):
        print("No difference found")
        #alert()
        continue
    else:
        print("The site has been updated")
        alert()
        shutil.copyfile("first_photo.png", "second_photo.png")
```

POSSIBLE EXTENSIONS

Where this could go from here

- Monitor several URLs in parallel, not just one
- Compare a specific region or CSS selector, not the whole page
- Tolerate small differences (a % threshold) instead of exact equality
- Keep a timestamped history of screenshots
- Replace the infinite loop with a scheduled job (cron / Docker)